Energy-Efficient Scheduling for Stability-Guaranteed Embedded Control Systems

Alireza Salami Abyaneh
School of Electrical and Computer Engineering,
University of Tehran, Iran
ar.salami@ut.ac.ir

Mehdi Kargahi
School of Electrical and Computer Engineering,
University of Tehran, Iran
and School of Computer Science, IPM
kargahi@ut.ac.ir

***Abstract*—Stability, which is heavily dependent on the controller delays, is the main measure of performance in embedded control systems. With the increased demand for resources in such systems, energy consumption has been converted to an important issue, especially in systems with limited energy sources like batteries. Accordingly, in addition to the traditional temporal requirements in these systems, stability and economic energy usage are further demands for the design of embedded control systems. For the latter demand, dynamic voltage and frequency scaling (DVFS) is too usual, however, as this technique increases the controller delay and jitter, it may negatively impact the system stability. This paper addresses the problem of control task priority assignment as well as task-specific processor voltage/frequency assignment such that the stability be guaranteed and the energy consumption be reduced. The proposed idea considers the task execution-time variability and increases the processor frequency only when the task execution-time exceeds some threshold. Experimental results show energy-efficiency of the proposed method for embedded control systems.**

***Index Terms*—Energy management, real-time scheduling, stability guarantee, embedded control systems, stochastic execution-time**

I. Introduction

Embedded control systems often consist of several algorithms implemented as periodic tasks for closed-loop control of physical plants. The tasks periodically perform plant sampling, control signal computation, and actuation. The main requirement in such systems is to preserve the stability of the control loops, where the stability can be determined by the tolerable amounts of delay and jitter [1, 2]; thus, it depends on the task timings. Concurrent execution of the tasks on a shared resource, however, may cause more variability in the delays and jitters, which if non properly taken into account can jeopardize stability of the control loops [3, 4].

The complexity of emerging embedded control systems is being increased [5], which in turn it leads to the increased demand for computing resources. Many such systems (like the things connected to the Internet of Things) are energy-limited devices, e.g., they are battery-operated with limited power and energy budgets. Thus, an important problem for the systems is to reduce the system energy consumption while still preserving the stability and quality-of-control (QoC) requirements. The main processor of many such control devices is the most energy consuming part [6].

Dynamic voltage and frequency scaling (DVFS) is a well-known technique for reducing energy consumption of the main processor in embedded and real-time systems. Energy management methods using DVFS has been widely studied in the real-time systems literature [7, 8], however, as DVFS works with the processor supply voltage and operating frequency, applying it to control systems is rather more complex from the aspect of stability, as described below.

The main idea behind employing DVFS in real-time systems is lowering the processor supply voltage/frequency in a task-specific manner as far as the tasks still meet their deadlines. However, in control systems, the stability of control loops is important, which can be expressed by the amount of delay and jitter that each control task can tolerate [1, 2]. Thus, in order to use DVFS for embedded control systems, both delay and response time jitter should be taken into account. Most of the works on DVFS in the context of real-time systems only consider total delay to be less than the relative deadline; in this study, however, we consider both delay and jitter. To the best of the authors' knowledge, we are unaware of any previous study on energy-efficient scheduling which cares about acceptable delays and jitters according to the control task stability constraint, while there exist some energy unaware methods [2, 9].

The current study presents a preemptive fixed-priority energy-efficient scheduling algorithm for a set of periodic control tasks with stochastic execution-times. The goal of the proposed algorithm is to reduce processor energy consumption, while guaranteeing the stability of all control loops in the system.

We propose a three-step offline scheduling algorithm. The first step is associated with developing an energy-efficient method for priority assignment to tasks. The second step is to present a DVFS schedule which reduces the maximum energy consumption of the system by exploiting static slacks. In the last step, however, the processing frequency of each task is reclaimed using the DVFS schedule obtained in the second step beside some probabilistic information about the task execution-times. The goal of applying the last step is to assign appropriate processing frequencies to the tasks based on the average workload in a manner that the expected energy consumption of the system be reduced. Although during peak loads, execution of tasks at a high processing frequency is es-

sential for preserving stability of corresponding control loops, this event may rarely occur [8, 10]. Further, designing the system according to the peak load causes wastage of energy consumption. Most of the times that average-case execution-times happen, there is no need to execute the tasks with high processing frequencies.

The remainder of this paper is organized as follows. Related work is discussed in Section II. The system model and problem definition are presented in Section III. The stability analysis of control tasks is described in Section IV. Next, Section V presents our energy-efficient scheduling method. Evaluation of the proposed method is studied in Section VI, and the paper is concluded in Section VII.

II. Related Work

The problem of reducing energy consumption using DVFS has been widely addressed in the embedded and real-time systems literature. There are many works on fixed-priority DVFS scheduling, e.g., [7, 11–13], however with no consideration of control system requirements like stability and control quality.

In the context of control systems, there are studies which consider both control performance and energy consumption using the DVFS technique [6, 14–17]. The scheduling policy assumed in these studies is Earliest Deadline First (EDF), namely a dynamic priority scheduling method. However, in many applications, fixed-priority scheduling algorithms may be preferred because of the following reasons: 1) implementation of such methods is relatively simple; 2) their runtime overhead is usually lower; and 3) task priorities can be determined as desired by the designer [18]. Hence, in this paper we focus on systems with fixed-priority scheduling and consider the effect of task priority on the system stability and energy consumption.

Most of the papers applying DVFS to control systems dynamically adjust the processing frequency of tasks. However, such algorithms may impose higher scheduling overheads due to their online decisions. Also, in the case of heavy workloads, online approaches often perform worse than offline methods from the energy consumption aspect [11]. Therefore, in this paper we present an offline processing frequency assignment method for a fixed-priority preemptive control system that considers effect of priorities on the energy-efficiency. Our DVFS scheduling method guarantees stability of each control loop while reducing the overall system energy consumption.

There also exist some studies which consider both jitter and DVFS scheduling in context of real-time systems. Mochocki et al. [19] consider meeting the completion-time jitter constraints of a set of periodic tasks scheduled by fixed-priority algorithms. They report that DVFS may cause increase in jitter when used aggressively; but if employed carefully, it can be used for satisfying the jitter constraints. They only consider one completion jitter constraint for each task; however, in this paper, which we consider both delay and jitter together, we have a set of acceptable delays and jitters known as the stability area for each task. Also, another important difference between their work and ours is that they assume that completion jitter is only due to preemptions and do not consider the effect of execution-time variability, what we consider in the current study. Also in [20], the authors discussed the impact of using DVFS on the task jitter, however they focus on non-preemptive real-time systems with low jitter design considerations.

III. System Model and Problem Definition

This section first describes the models of control tasks and processor power consumption, and then defines the considered problem.

A. Task Model

We consider a set of n periodic real-time control tasks, denoted as $\tau = \{\tau_1, \tau_2, ..., \tau_n\}$, which are executed on a single processor with some fixed-priority preemptive scheduling policy. Each task is denoted by a tuple $\tau_i = (p_i, T_i, C_i, D_i, \phi_i)$, where p_i, T_i, C_i, D_i, and ϕ_i specify priority, period, execution-time, relative deadline, and the required level of stability for task τ_i, respectively. We assume that tasks with smaller values of p_i have higher priorities. Stability requirement of each task, ϕ_i, is expressed based on the notion *apparent phase margin*, that will be introduced in Section IV.B. For each task, execution-time C_i, is a random variable with probability density function $pdf_i(.)$ which is bounded by the best-case execution-time C_i^b and the worst-case execution-time C_i^w. It is assumed that the relative deadline of each task instantiation (also known as job) is less than or equal to the respective task period.

Each task controls a plant in a closed loop. Control tasks periodically sample the plant, compute the control signal, and actuate that signal to the plant. Each plant is described by the linear continuous-time differential equations as:

$$\dot{x}_i = \mathfrak{A}_i x_i + \mathfrak{B}_i u_i, \quad y_i = \mathfrak{C}_i x_i, \tag{1}$$

where x_i, y_i, and u_i are the state vector, the output of the plant, and the applied control signal, respectively. Also $\mathfrak{A}_i$, $\mathfrak{B}_i$, and $\mathfrak{C}_i$ are the state, input, and output matrices describing the dynamics of the plant. The output of the plant is sampled with the constant interval of T_i. In each sampling interval, control signal is updated after some time-varying delay and it is held constant until the next update.

B. Power Model

The power consumption of the processor at frequency f_i is characterized by following equations [8, 21]:

$$P(f_i) = P_s + h(P_{ind} + P_d), \quad P_d = \gamma . f_i^m, \tag{2}$$

where P_s, P_{ind}, and P_d are static power, frequency-independent active power, and frequency-dependent active power, respectively. When system is active $h = 1$, otherwise, $h = 0$. γ, and m ($2 \leq m \leq 3$) are the switch capacitance, and the dynamic power exponent which are system-dependent constants.

We assume processor is capable of operating at frequencies in range $[f_{min}, f_{max}]$. All the task execution-times and the respective probability density functions have been defined

based on f_{max} in the task model. Also, we assume that the maximum frequency, f_{max}, is normalized to 1. By using DVFS we can lower processing frequency of a task as much as possible such that the timing and stability constraints are not violated. However, if this frequency falls below a threshold, the task consumes more static and frequency-independent active energy. The value of the threshold can be computed as $f_{th} = \sqrt[m]{\frac{P_{ind}}{\gamma.(m-1)}}$. Therefore, minimum possible operating frequency of processor, f_{min}, should be greater than f_{th} [21].

Using DVFS, this paper only focuses on reducing the dynamic energy, computed as $E = t.P_d$ for CMOS circuits, through appropriate CPU frequency assignment. Therefore, the energy consumption for a task τ_i in a hyperperiod Π, will be:

$$E_i = \frac{\Pi}{T_i}.\frac{C_i}{f_i}.P_d = \frac{\Pi}{T_i}.C_i.\gamma.f_i^{m-1}. \tag{3}$$

C. Problem Definition

In the remainder of this paper, we study the problem of energy-efficient scheduling of an embedded control system with the proposed model; the goal is to present an algorithm to determine the priorities and task processing frequencies such that stability requirement ϕ_i of each control task be guaranteed, and to run the system in an energy-efficient manner. More formally, we have a set of periodic real-time control tasks, $\tau = \{\tau_1, \tau_2, ..., \tau_n\}$, that each control a plant in a closed loop. Corresponding to each control task there exists a required level of stability denoted by $\phi_i \in \Phi$, where Φ specifies the set of stability requirements. The goal is to choose the priority p_i and the processing frequency $f_i \in [f_{min}, f_{max}]$ for each task τ_i such that the energy consumption of tasks be reduced, while all the stability requirements given by the set Φ be guaranteed.

IV. Stability Analysis

In this section, we first review the stability analysis of a control system and describe how to extract the requirements to preserve stability of a plant when we have variable input-output delays. Then, the stability criteria used in this work is introduced.

A. Stability Margin

Consider a closed control loop with a linear continuous-time plant, a linear discrete-time controller and time-varying input-output delays. The input-output delay experienced by the controller can be divided into two parts: a constant part, L, and a time-varying part or jitter, J. Hence, the minimum and maximum possible delays are given respectively by L and $L + J$ [1]. Stability of such closed control loop can be determined by the amounts of minimum delay and jitter the control task can tolerate. This can be achieved by using the Jitter Margin toolbox [1, 22]. In fact, the output of Jitter Margin toolbox is a stability curve denoted by $JM(L)$, that gives the maximum tolerable jitter as a function of the constant part of delay, L. One primary property of Jitter Margin [1] has been given below:

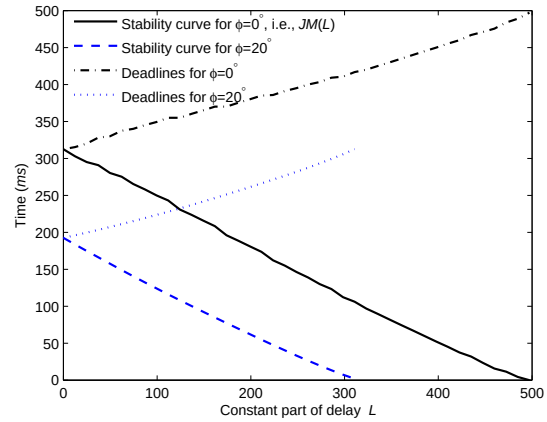

Fig. 1: Sample stability and deadline curves generated by the Jitter Margin toolbox

Property 1. *$L + JM(L)$ is an increasing function of L.*

This implies that if the constant part of delay increases for t units, the tolerable jitter decreases less than t units.

B. Stability Criteria

Stability requirements of each control task is determined by the controller designer in terms of the required *apparent phase margin* denoted as ϕ [1]. Considering the minimum delay L and the jitter J, the *apparent phase margin* is defined with respect to the stability curve $JM(L)$, as the smallest ϕ that solves $J = JM(L + \frac{\phi}{\omega})$, where ω is the crossover frequency of control loop for constant time delay of L. The system stability is guaranteed if the value of ϕ be greater than or equal to zero; any values of greater than zero is interpreted as a level of stability guarantee [1].

For a given stability curve $JM(L)$ and specified values of L and ϕ, the tolerable amount of jitter can be calculated as:

$$JM(L + \frac{\phi}{\omega}). \tag{4}$$

The required level of stability for a task is guaranteed, if its response time jitter J be less than or equal to the tolerable jitter achieved based on (4). Therefore, the stability curve for a specified $\phi \geq 0$ can easily be derived using (4) for different values of L. Accordingly, by using the Jitter Margin toolbox, the hard deadline of a task which guarantees its required level of stability can be calculated as:

$$D = L + JM(L + \frac{\phi}{\omega}). \tag{5}$$

Based on Property 1, when the value of L is increased, the tolerable amount of deadline is also increased.

As an example, consider the plant $\frac{1}{s^2+2\zeta\xi s+\xi^2}$, with $\zeta = 0.7$, $\xi = 0.9$ and a LQG controller with period $500ms$; the stability and hard deadline curves for ϕ values of $0°$ and $20°$ are shown in Fig. 1. All points (L, J) on and below the stability curve build the stability area for the specified control task. For simplicity, we can approximate the stability curve corresponding to the task by a linear function of $J + \alpha L = \beta$ [2]. Thus, for a given task τ_i, we can check whether the pair (L_i, J_i) showing its constant part of delay and jitter in a given

schedule, falls within the stability area or not, by checking the inequality $J_i + \alpha_i L_i \leq \beta_i$.

To calculate the minimum delay and jitter experienced by a control task, we use the following response time analysis. Let R_i^w and R_i^b denote, respectively, the worst-case and best-case response times of task τ_i. The constant part of delay, L_i, and the jitter, J_i, are then given by,

$$L_i = R_i^b, \tag{6}$$

$$J_i = R_i^w - R_i^b. \tag{7}$$

In the following, we present how to calculate the worst-case and best-case response times under fixed-priority preemptive scheduling, assuming deadline $D_i \leq T_i$.

The exact worst-case response time for task τ_i is calculated using the following equation [23]:

$$R_i^w = C_i^w + \sum_{\tau_j \in hp(\tau_i)} \lceil R_i^w / T_j \rceil C_j^w, \tag{8}$$

where $hp(\tau_i)$ denotes the task set with higher priorities with respect to task τ_i. Also, an upper bound for the worst-case response time of task τ_i can be obtained from the following equation [2, 24]:

$$\overline{R}_i^w = \frac{C_i^w + \sum_{\tau_j \in hp(\tau_i)} C_j^w (1 - u_j^w)}{1 - \sum_{\tau_j \in hp(\tau_i)} u_j^w}, \tag{9}$$

where $u_i^w = \frac{C_i^w}{T_i}$ is the worst-case utilization of τ_i.

The exact best-case response time for task τ_i is calculated as [25]:

$$R_i^b = C_i^b + \sum_{\tau_j \in hp(\tau_i)} \lceil (R_i^b / T_j) - 1 \rceil C_j^b. \tag{10}$$

A lower bound for the best-case response time also is derived as follows [2]:

$$\underline{R}_i^b = max \left\{ \frac{C_i^b - \sum_{\tau_j \in hp(\tau_i)} C_j^b (1 - u_j^b)}{1 - \sum_{\tau_j \in hp(\tau_i)} u_j^b}, C_i^b \right\}, \tag{11}$$

where $u_i^b = \frac{C_i^b}{T_i}$ denotes the best-case utilization of τ_i.

V. The Proposed Energy-Efficient Scheduling Method

We employ DVFS for the proposed energy-efficient processing frequency assignment method. The method first assigns task priorities based on an energy-efficient algorithm, while preserving the task stabilities. Then, it uses static slacks for offline frequency assignment in two steps: 1) Reducing the maximum energy consumption by using the priority monotonic frequency assignment approach, and 2) Workload-dependent frequency assignment which uses the frequencies of the previous step and reduces the expected energy consumption of the control tasks. Using this method, the control loop stabilities, even in the worse scenarios (i.e., when the tasks exhibit their worst-case execution-times), are guaranteed.

A. Priority Assignment

In order to assign task priorities, we follow two objectives: 1) guaranteeing the stability requirement ϕ_i of each task τ_i, and 2) energy-efficiency of the priority assignment method. The first objective is fulfilled by assigning higher priorities to tasks which are more sensitive to delay and jitter. In this regard, Aminifar et al. [2] discussed the priority assignment problem in real-time control systems, and presented an optimal algorithm that finds some priority orders which satisfies the stability requirements, determined by the stability area of each task (which has been described in Section IV). The algorithm divides the tasks into some priority groups and guarantees the corresponding stabilities if the relative order between the groups is respected. Their study tells that increasing priority does not necessarily lead to jitter reduction, when it is calculated using (7), (8), and (10). For ease of priority assignment analysis, they proposed the following equation to calculate the jitter:

$$J_i = \overline{R}_i^w - \underline{R}_i^b. \tag{12}$$

Using the above equation makes jitter calculation pessimistic, whereas it provides a monotonic behavior between jitter and priority. According to their study, from the stability perspective, it is safe to use the bounds on worst-case and best-case response times for obtaining the jitter values.

Therefore, in our proposed method, we first categorize the tasks in some priority groups $G = [g_1, g_2, \ldots, g_i]$, according to the algorithm of [2]; then, in order to satisfy the second objective, in each of these groups, prioritize the tasks such that their *scaling factor* with the following definition is minimized.

Definition 1. *The scaling factor of a task set S is the minimum frequency f such that if all execution-times C_i are replaced by $\frac{C_i}{f}$, still the stability requirements of the tasks be preserved.*

Algorithm 1 shows our energy-efficient priority assignment method. This algorithm gets priority groups $G = [g_1, g_2, \ldots, g_i]$, which are sorted based on descending order of priorities, and assigns a unique priority to each task. According

Algorithm 1: Priority Assignment

```
Input  : G = [g_1, ..., g_i], ϵ  // ϵ is precision of calculations
Output: P = [p_1, p_2, ..., p_n]  // task priorities
p = n;
for j = i downto 1 do
    HP = ⋃_{z=1:j} g_z;
    while g_j ≠ null do
        foreach τ_k in g_j do
            hp(τ_k) = HP \ τ_k;
            f_k = MinFrequency(hp(τ_k), τ_k, f_min, f_max, ϵ);
        end
        τ_m ← find τ_k ∈ g_j having minimum f_k;
        p_m ← p;  // assign priority of p to τ_m
        p = p − 1;
        g_j = g_j \ τ_m;
        HP = HP \ τ_m;
    end
end
//
function MinFrequency(hp(τ_k), τ_k, f_down, f_up, ϵ)
while f_up − f_down > ϵ do
    f_tmp ← (f_down + f_up)/2;
    • Calculate L_k and J_k of task τ_k based on (6) and (12), considering f_tmp
      as processing frequency of tasks in hp(τ_k) ⋃ τ_k;
    if J_k + α_k L_k ≤ β_k then
        f_up ← f_tmp;
    else
        f_down ← f_tmp;
    end
end
return f_up
end function
```

to the algorithm, between different priority orders of each group, one order that is more energy-efficient is chosen. In each iteration of the main loop (Lines 2 to 15), the tasks in the highest priority group are prioritized as explained in the following. From Line 5 to Line 8, for each task $\tau_k \in g_j$, we assume that higher priority tasks are $hp(\tau_k) = HP \backslash \tau_k$, where HP is the set of all tasks which are located in groups with priority higher than or equal to g_j. Then, we calculate the minimum tolerable frequency of τ_k, i.e., f_k, by using the `MinFrequency` function. `MinFrequency` returns the minimum processing frequency of tasks placed in the set of HP, so that the task τ_k remains stable. Inside of this function we only focus on checking stability of τ_k, so it is not necessary to know the priority order of tasks in $hp(\tau_k)$. After calculation of f_k for each task $\tau_k \in g_j$, between all tasks in the current group g_j, priority p is given to the task which tolerates the relatively lowest frequency; then, it is removed from the g_j and HP sets, and p is updated (Lines 9 to 13). The same procedure is continued until all tasks take their unique priorities.

B. Reducing the Maximum Energy (RME)

The problem of finding an energy-optimal fixed-priority voltage schedule for real-time tasks is NP-hard [12]. However, there exist several fixed-priority frequency scheduling heuristics for the problem, e.g., [11–13, 26]. One of these heuristics is priority monotonic frequency assignment [11, 13]. This heuristic works in an energy-efficient manner because of the following reason. In a fixed-priority real-time system, the execution of each task is influenced by the higher priority tasks. Thus, assigning low processing frequencies to higher priority tasks imposes use of high frequencies for lower priority tasks. This approach may not be energy-efficient; also, the convexity of power function proposes using of the same frequency for tasks of different priorities, or at least, if different frequencies are required, imposes using frequencies which are close to each other as much as possible. This objective helps to reduce the maximum frequency needed for executing the tasks.

In [11], Perathoner et al. show the way to use priority monotonic approach for frequency assignment in a fixed-priority real-time system with non-deterministic release times. Their heuristic satisfies the following properties:

1) Higher priority tasks should be executed by higher processing frequencies with respect to lower priority tasks.
2) The processing frequency of lower priority tasks can be scaled down more than higher priority tasks, only when the frequency of higher priority tasks cannot be reduced more. This implies that as far as possible, the processing frequencies should be close together.

In the control system considered in this paper, in order to fulfill the above properties, first a scaling factor (Definition 1) for the entire task set τ, should be determined. Then, the task with priority i which prevents further reduction of the scaling factor must be identified and named as the *bottleneck task*. The *bottleneck task* prevents more reduction of the scaling factor because if it is reduced more, the task violates its stability requirement. After determination of the scaling factor for the whole task set and finding the bottleneck task with priority i, processing frequency of tasks with priorities 1 to i is set to the value of scaling factor and then this procedure is continued for the remaining tasks with priorities $i+1$ to n. We adapt the abovementioned heuristic for the considered control system with the goal of reducing the maximum energy consumption.

In order to find the scaling factor for the task set $S \subseteq \tau$, for each task $\tau_i \in S$, we need to check that, regarding the constant part of delay L_i and jitter J_i, the task is still inside its stability area (see Fig. 1). Calculation of J_i using (12) makes the analysis pessimistic, thus determination of the frequencies based on which is also pessimistic. Therefore, according to L_i and D_i which are calculated based on (6) and (5), we propose a frequency assignment algorithm which only uses deadlines that guarantee stability (see Fig. 1), and does not take the jitters into account. Algorithm 2 summarizes the proposed frequency assignment method which has been adopted for the considered control system. This algorithm gets the entire task set, sorted in descending order of priorities, and returns the appropriate task processing frequencies.

Algorithm 2: Processing Frequency Selection

Input : τ, n, ϵ `// $\tau$ is the task set sorted in descending order of priorities, $n$ is number of tasks, $\epsilon$ is precision of calculations`
Output: $F^{RME} = [f_1^{RME}, f_2^{RME}, \ldots, f_n^{RME}]$

```
1  D = [D_1 = T_1, D_2 = T_2, ..., D_n = T_n]; // Deadlines
2  F^RME = [f_1^RME = f_max, f_2^RME = f_max, ..., f_n^RME = f_max];
3  f_scale = f_max;
4  for i = 1 to n do
5      repeat
6          for j = i to n do
7              L_j ← Constant part of delay calculation, considering F^RME
                 frequencies and according to (6);
8              D_j ← Deadline calculation according to (5);
9          end
10         f_scale_prev = f_scale;
11         f_scale = ScaleFactor(i, n, f_scale_prev, D, ε);
           // applying new frequencies
12         F^RME(i : n) = f_scale;
13     until f_scale_prev − f_scale > ε;
14 end
```

The main loop of Algorithm 2 is between Lines 4 and 14, where the scaling factor is computed for each subset of τ from priority level i to n. In fact, by computing the scaling factor for each of these subsets, each task is tested in the position of being the bottleneck task. An iterative procedure is employed (Lines 5 to 13) in order to determine scaling factor of each task subset. In each iteration of this procedure, first from Line 6 to Line 9, each task deadline is assigned based on (5). If the calculated relative deadline is greater than the task period, we assign the period as the task relative deadline. Then, for these tasks with the specified deadlines, the amount of scaling factor is calculated using the `ScaleFactor` function (Algorithm 3). Because the calculated scaling factor is non-increasing in consecutive iterations, by applying the calculated factors to the tasks, the constant parts of delay are not decreased and therefore based on Property 1, the deadlines are also not decreased. This procedure is continued until finding that there

is no further gain in reducing the scaling factor by assigning new deadlines.

The scaling factor is calculated by using the binary search method inside the `ScaleFactor` function, where for each specified frequency, feasibility of tasks from priority i to n is investigated using `Stable`(i, n, f_{tmp}, D). As the total delay, i.e., $L + J$, has a monotonic behavior with respect to the processing frequency, using the search method is thus safe and returns the minimum scaling factor. Inside of `Stable` function, the worst-case response times of tasks from priority i to n are checked to be less than or equal to their relative deadlines D, considering f_{tmp} as their processing frequencies. The worst-case response times are also calculated according to (8).

Algorithm 3: ScaleFactor

```
Input  : i, n, f, D, ε
Output: f_up
f_down ← f;
f_up ← f_max;
while f_up − f_down > ε do
    f_tmp ← (f_down + f_up)/2;
    if Stable(i, n, f_tmp, D) == true then
        f_up ← f_tmp;
    else
        f_down ← f_tmp;
    end
end
```

C. Reducing the Expected Energy (REE)

In this step, we focus on variability of the workloads. More precisely, we show how to use the task probability distribution function of execution-times to reduce the system expected energy consumption. Since occurrence of worst-case execution-time is rare and tasks usually run for less than their worst-case execution-time [8, 10], acquiring frequencies according to the worst-case workload is pessimistic. Hence, we can choose the processing frequencies based on average-case workload; then, if this case has been violated, we must have the chance to speedup the processor to handle even worst-case execution-times. In this regard, a common technique is to split the task execution-time into two parts, and assign different processing frequencies to the parts, in a manner that the processing frequency of the first part be lower than that of the second part [10, 27]. Since the chance of having the task execution-time in the first part is higher, this approach helps to reduce the task expected energy consumption.

We split the execution-time of the task τ_i into two parts: the first part which is executed at $f_i^x \leq f_i^{RME}$ (f_i^{RME} is the output of Algorithm 2), and the second part that is executed at frequency of f_{max}. The point at which the processing frequency of τ_i is changed from f_i^x to f_{max} is defined as the *transition point*, x_i. By executing the second part at frequency f_{max}, some slack time is created, which we use it to run the first part at a lower frequency. Thus, we should find the following parameters for each task:

- x_i, the transition point for task τ_i,
- f_i^x, processing frequency of the first part of task τ_i.

In order to choose the transition point of a task, we assume x_i as a point in execution interval of task τ_i ($x_i \in [C_i^b, C_i^w]$)

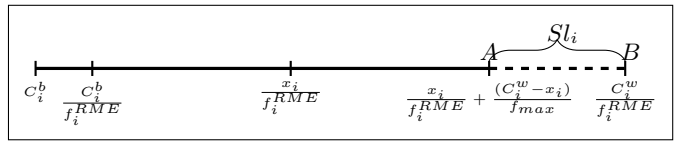

Fig. 2: The slack Sl_i has been generated from execution of the task second part at f_{max}

such that, if $C_i \leq x_i$, τ_i is executed at frequency f_i^x; and if $C_i > x_i$, the first x_i units of τ_i are run at frequency f_i^x, then, the remaining $C_i - x_i$ units are executed at frequency f_{max}. Therefore, the energy consumption of task τ_i is calculated as:

$$E_i = \begin{cases} cdf_i(x_i).\dfrac{\Pi}{T_i}.x_i.\gamma.(f_i^x)^{m-1} & C_i \leq x_i \\ (1 - cdf_i(x_i)).\dfrac{\Pi}{T_i}.\gamma.[x_i.(f_i^x)^{m-1} & \\ + (C_i^w - x_i).f_{max}^{m-1}] & C_i > x_i \end{cases} \tag{13}$$

where $cdf_i(.)$ is the cumulative distribution function of task τ_i execution-time.

Using the frequencies F^{RME}, obtained in the previous section, f_i^x can be written as terms of x_i, C_i^w, and f_i^{RME}. As shown in Fig. 2, the dashed line shows the slack, Sl_i, resulting from fast execution of task after its transition point. This slack can be used for slowing down the execution of the first part, namely the part before the transition point. The length of generated slack can be determined according to points A and B in Fig. 2. The former point denotes the worst-case execution-time for the scenario that the first x_i units of execution are run at f_i^{RME} and the remaining $C_i^w - x_i$ units are executed at f_{max}. The latter point shows the worst-case execution-time of the task, when the whole task is executed at f_i^{RME}. Thus, the length of Sl_i is determined as follows:

$$Sl_i = \frac{C_i^w}{f_i^{RME}} - \frac{x_i}{f_i^{RME}} - \frac{(C_i^w - x_i)}{f_{max}}. \tag{14}$$

By applying the slack Sl_i to the first part of task execution, the processing frequency f_i^x, can be obtained as:

$$f_i^x = \frac{x_i}{\frac{x_i}{f_i^{RME}} + Sl_i} = \frac{x_i}{\frac{C_i^w}{f_i^{RME}} - \frac{(C_i^w - x_i)}{f_{max}}}. \tag{15}$$

Finally, the energy consumption of task τ_i, is calculated as follows:

$$E_i = \frac{\Pi}{T_i}.\gamma.x_i.\left(\frac{x_i}{\frac{C_i^w}{f_i^{RME}} - \frac{(C_i^w - x_i)}{f_{max}}}\right)^{m-1} + \frac{\Pi}{T_i}.\gamma.(1 - cdf_i(x_i)).(C_i^w - x_i).f_{max}^{m-1}. \tag{16}$$

The energy consumption E_i, is a function of x_i, and we can determine x_i in a manner that E_i is minimized. By substituting the extracted transition point x_i in (15), we can calculate the desired frequency of each task which the processor should run at it before the transition point. This way, we have two processing frequencies for each task, denoted as $f_i = (f_i^x, f_{max})$, in which f_i^x and f_{max} are frequencies for the first and second parts of the τ_i's execution. Therefore, the output of our processing frequency assignment method is as $F = [f_1 = (f_1^x, f_{max}), f_2 = (f_2^x, f_{max}), ..., f_n = (f_n^x, f_{max})]$.

Theorem 1. *By using the REE method mentioned in Section V.C, the stability requirements of all tasks are preserved. In other words, the new delay and jitter resulted for each task from applying the new frequencies F, remain in the task stability area.*

Proof. Using the *REE* method, the worst-case response time of task τ_i, i.e., R_i^w, remains unchanged in comparison with the *RME* method proposed in Section V.B. Only the best-case response time R_i^b, may be increased. If R_i^b increases for b units, the amount of L_i increases for b units as well, and thus the jitter J_i decreases for b units. Therefore, according to Property 1, we conclude that the changed amounts of delay and jitter for each task always maintain the task within its stability area. □

VI. Experiment Results

In this section, we evaluate our energy-efficient scheduling method. In this regard, the task and plant sets are randomly generated, where the plants have been chosen from the set given in [28, 29]. Corresponding to each plant we consider a control task. In order to generate the task sets, first for a given utilization, random utilization values u_i are generated using the UUniFast method [30]. Then, for a specified task τ_i, the worst-case execution-time is selected randomly within the range of $C_i^w \in U(0.01, 0.02)$. Also, the best-case execution-time C_i^b is assumed to be randomly chosen from 20% to 40% of the selected worst-case execution-time. The probability density function of execution-time for the task is randomly selected from Normal and Gamma distributions which have been bounded by the best-case and worst-case execution-times. Period of each random task τ_i is obtained by calculating the ratio of C_i^w and u_i.

In order to determine parameters of the power model (2), we consider the following values: $P_s = 0$, $P_{ind} = 0.08$, $\gamma = 1.52$ and $m = 3$ [11]. Also, we assume that f_{max} is normalized to 1. Thus, the value of threshold processing frequency can be calculated as $f_{th} = 0.297$ and the minimum available frequency is set to $f_{min} = 0.3$ ($> f_{th}$).

To evaluate energy consumption of our DVFS scheme, we set up system utilizations from 0.5 to 0.9 with steps of 0.1. For each utilization, we construct 1000 test cases consisting of $n = \{3, 5\}$ control tasks, each of them controlling a plant. We assume that the required level of stability for all of the task sets is $\Phi = \{\phi_1 = 0, \phi_2 = 0, \ldots, \phi_n = 0\}$. The expected energy consumption of each test case is evaluated for time interval of 1000 second and the value of ϵ is set to 10^{-4}.

In the experiment, we compare the expected energy consumption of test cases when the following methods are applied:

- The energy-efficient stability-guaranteed scheduling (EESGS) method proposed in Section V.
- The priority assignment (PA) method presented in Algorithm 1 without using DVFS.
- The combined use of Algorithm 1 and 2 (PA-RME).
- The algorithm that uniformly scales down all the task processing frequencies; this algorithm assigns only one minimized frequency to the whole task set such that the task stability requirements are not violated (Uniform).
- The energy-unaware priority assignment (EUPA) method used in [2], combined with the frequency assignment method presented in Sections V.B and V.C.
- The energy-efficient scheduling method proposed in [11] (EERTS).

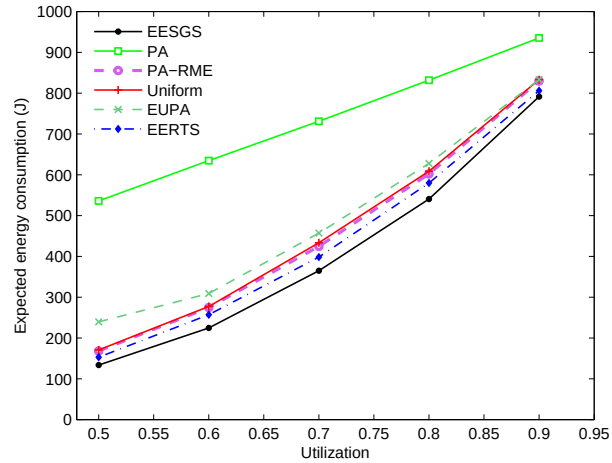

Fig. 3: Expected energy consumption of our proposed method, EESGS, compared with other methods.

Fig. 3 reports the expected energy consumption using EESGS compared with PA, PA-RME, Uniform, EUPA, and EERTS methods, for different utilizations. The results show that our energy-efficient scheduling method can save more energy without any stability violation. As shown in the figure, assigning processing frequencies using the EESGS method is more effective than uniformly scaling all tasks by the same factor (Uniform). We have gained on average 14.56 % improvement in terms of energy consumption compared with the Uniform approach. Also in comparison between EESGS and EUPA methods, the effectiveness of our priority assignment in energy saving is illustrated when priorities are assigned based on Algorithm 1 and the optimal priority assignment method of EUPA. The proposed method in [2] considers stability requirement of each task, however, our priority assignment method assigns priorities based on both stability and energy consumption of the task set. Employing our priority assignment method saves an average 22.14 % of the energy over the EUPA method, as shown in the figure.

Finally, the EERTS method given in [11] is compared against our proposed EESGS method, when we have a control system with stability requirements. The EERTS method, presents a DVFS scheme for a hard real-time system without stability constraints. In this experiment, applying the EESGS method has on an average 8.45 % energy saving compared with EERTS. Nevertheless, as the curve PA-RME in Fig. 3 indicates, if only the second step of EESGF, i.e. RME, is employed for processing frequency assignment (without use of REE), our method cannot work better than EERTS in energy saving. This is because in our control system, tasks are constrained by stability requirements in addition to deadlines and thus processing frequencies may lesser be reduced. However,

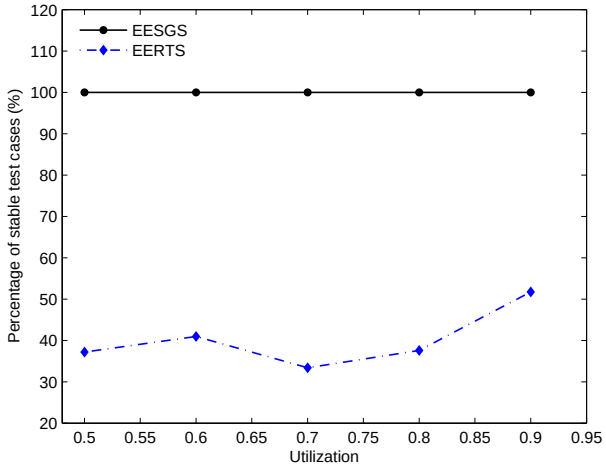

Fig. 4: Percentage of test cases which are stability-guaranteed in comparison of our method EESGS and EERTS.

applying EERTS method may violate stability requirements of the task set. Fig. 4 shows percentage of test cases which are stability-guaranteed. The EERTS method can guarantee an average 40.18 % of the test cases, whereas, our method guarantees all of them.

VII. Conclusion

This paper discusses the energy reduction of a fixed-priority embedded control system consisting of a set of tasks with stability requirements. We showed that applying DVFS in such systems, affects the delay and jitter experienced by each control task. Since from the stability perspective, each task can tolerate some specified amounts of delay and jitter, we proposed a scheduling method that considers these parameters together. To this end, we assigned task priorities and processing frequencies such that the stability requirements of all tasks are guaranteed while energy consumption is reduced. The expected energy consumption of the system has been reduced by assigning appropriate task processing frequencies, regarding the tasks execution-time distributions. Simulation results show that our proposed energy-efficient stability-guaranteed scheduling method outperforms the other considered energy reduction methods.

References

[1] A. Cervin, B. Lincoln, J. Eker, K.-E. Arzén, and G. Buttazzo, "The jitter margin and its application in the design of real-time control systems," in *Proceedings of the 10th International Conference on Real-Time and Embedded Computing Systems and Applications*, 2004, pp. 1–9.

[2] A. Aminifar, P. Eles, Z. Peng, and A. Cervin, "Stability-aware analysis and design of embedded control systems," in *Proceedings of the 11th ACM International Conference on Embedded Software*, 2013, p. 23.

[3] G. Buttazzo and A. Cervin, "Comparative assessment and evaluation of jitter control methods," 2007, pp. 163–172.

[4] M. Nasri, M. Kargahi, and M. Mohaqeqi, "Scheduling of accuracy-constrained real-time systems in dynamic environments," *Embedded Systems Letters*, vol. 4, no. 3, pp. 61–64, 2012.

[5] A. Cervin, D. Henriksson, B. Lincoln, J. Eker, and K.-E. Årzén, "How does control timing affect performance?" *IEEE control systems magazine*, vol. 23, no. 3, pp. 16–30, 2003.

[6] H. S. Lee and B. K. Kim, "Coscheduling of processor voltage and control task period for energy-efficient control systems," *ACM Transactions on Embedded Computing Systems*, vol. 9, no. 3, p. 15, 2010.

[7] J.-J. Chen and C.-F. Kuo, "Energy-efficient scheduling for real-time systems on dynamic voltage scaling (dvs) platforms," in *RTCSA*, 2007, pp. 28–38.

[8] P. Huang, P. Kumar, G. Giannopoulou, and L. Thiele, "Energy efficient dvfs scheduling for mixed-criticality systems," in *Proceedings of the 14th International Conference on Embedded Software*, 2014, pp. 1–10.

[9] G. M. Mancuso, E. Bini, and G. Pannocchia, "Optimal priority assignment to control tasks," *ACM Transactions on Embedded Computing Systems*, vol. 13, no. 5s, p. 161, 2014.

[10] E. Bini and C. Scordino, "Optimal two-level speed assignment for real-time systems," *International Journal of Embedded Systems*, vol. 4, no. 2, pp. 101–111, 2009.

[11] S. Perathoner, L. Thiele, and J.-J. Chen, "Energy-efficient static priority and speed assignment for real-time tasks with non-deterministic release times," in *IEEE 16th International Conference on Embedded and Real-Time Computing Systems and Applications*, 2010, pp. 173–182.

[12] H.-S. Yun and J. Kim, "On energy-optimal voltage scheduling for fixed-priority hard real-time systems," *ACM Transactions on Embedded Computing Systems*, vol. 2, no. 3, pp. 393–430, 2003.

[13] S. Saewong and R. Rajkumar, "Practical voltage-scaling for fixed-priority rt-systems," in *Proceedings of the 9th IEEE Real-Time and Embedded Technology and Applications Symposium*, 2003, pp. 106–114.

[14] H. S. Lee and B. K. Kim, "Dynamic voltage scaling for digital control system implementation," *Real-Time Systems*, vol. 29, no. 2-3, pp. 263–280, 2005.

[15] W. Zhao and F. Xia, "An efficient approach to energy saving in microcontrollers," in *Advances in Computer Systems Architecture*, 2006, pp. 595–601.

[16] F. Xia and Y. Sun, "An enhanced dynamic voltage scaling scheme for energy-efficient embedded real-time control systems," in *Computational Science and Its Applications*, 2006, pp. 539–548.

[17] F. Xia, X. Dai, X. Wang, and Y. Sun, "Feedback scheduling of real-time control tasks in power-aware embedded systems," in *Second International Conference on Embedded Software and System*, 2005, pp. 513–518.

[18] G. Quan and X. Hu, "Enhanced fixed-priority scheduling with (m, k)-firm guarantee," in *Proceedings of the 21st IEEE Real-Time Systems Symposium*, 2000, pp. 79–88.

[19] B. Mochocki, X. S. Hu, R. Racu, and R. Ernst, "Dynamic voltage scaling for the schedulability of jitter-constrained real-time embedded systems," in *IEEE/ACM International Conference on Computer-Aided Design*, 2005, pp. 446–449.

[20] T. Phatrapornnant and M. J. Pont, "Reducing jitter in embedded systems employing a time-triggered software architecture and dynamic voltage scaling," *IEEE Transactions on Computers*, vol. 55, no. 2, pp. 113–124, 2006.

[21] D. Zhu and H. Aydin, "Reliability-aware energy management for periodic real-time tasks," *IEEE Transactions on Computers*, vol. 58, no. 10, pp. 1382–1397, 2009.

[22] A. Cervin, "Stability and worst-case performance analysis of sampled-data control systems with input and output jitter," in *American Control Conference*, 2012, pp. 3760–3765.

[23] M. Joseph and P. Pandya, "Finding response times in a real-time system," *The Computer Journal*, vol. 29, no. 5, pp. 390–395, 1986.

[24] E. Bini, T. H. C. Nguyen, P. Richard, and S. K. Baruah, "A response-time bound in fixed-priority scheduling with arbitrary deadlines," *IEEE Transactions on Computers*, vol. 58, no. 2, pp. 279–286, 2009.

[25] O. Redell and M. Sanfridson, "Exact best-case response time analysis of fixed priority scheduled tasks," in *Proceedings of the 14th Euromicro Conference on Real-Time Systems*, 2002, pp. 165–172.

[26] G. Quan and X. S. Hu, "Energy efficient dvs schedule for fixed-priority real-time systems," *ACM Transactions on Embedded Computing Systems*, vol. 6, no. 4, p. 29, 2007.

[27] J. R. Lorch and A. J. Smith, "Improving dynamic voltage scaling algorithms with pace," in *ACM SIGMETRICS Performance Evaluation Review*, vol. 29, no. 1, 2001, pp. 50–61.

[28] E. Bini and A. Cervin, "Delay-aware period assignment in control systems," in *Real-Time Systems Symposium*, 2008, pp. 291–300.

[29] M. Lluesma, A. Cervin, P. Balbastre, I. Ripoll, and A. Crespo, "Jitter evaluation of real-time control systems," in *Proceedings of the 12th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications*, 2006, pp. 257–260.

[30] E. Bini and G. C. Buttazzo, "Measuring the performance of schedulability tests," *Real-Time Systems*, vol. 30, no. 1-2, pp. 129–154, 2005.